JavaScript DOM 编程

Web 开 发 基 础

by

计算机科学与技术学院

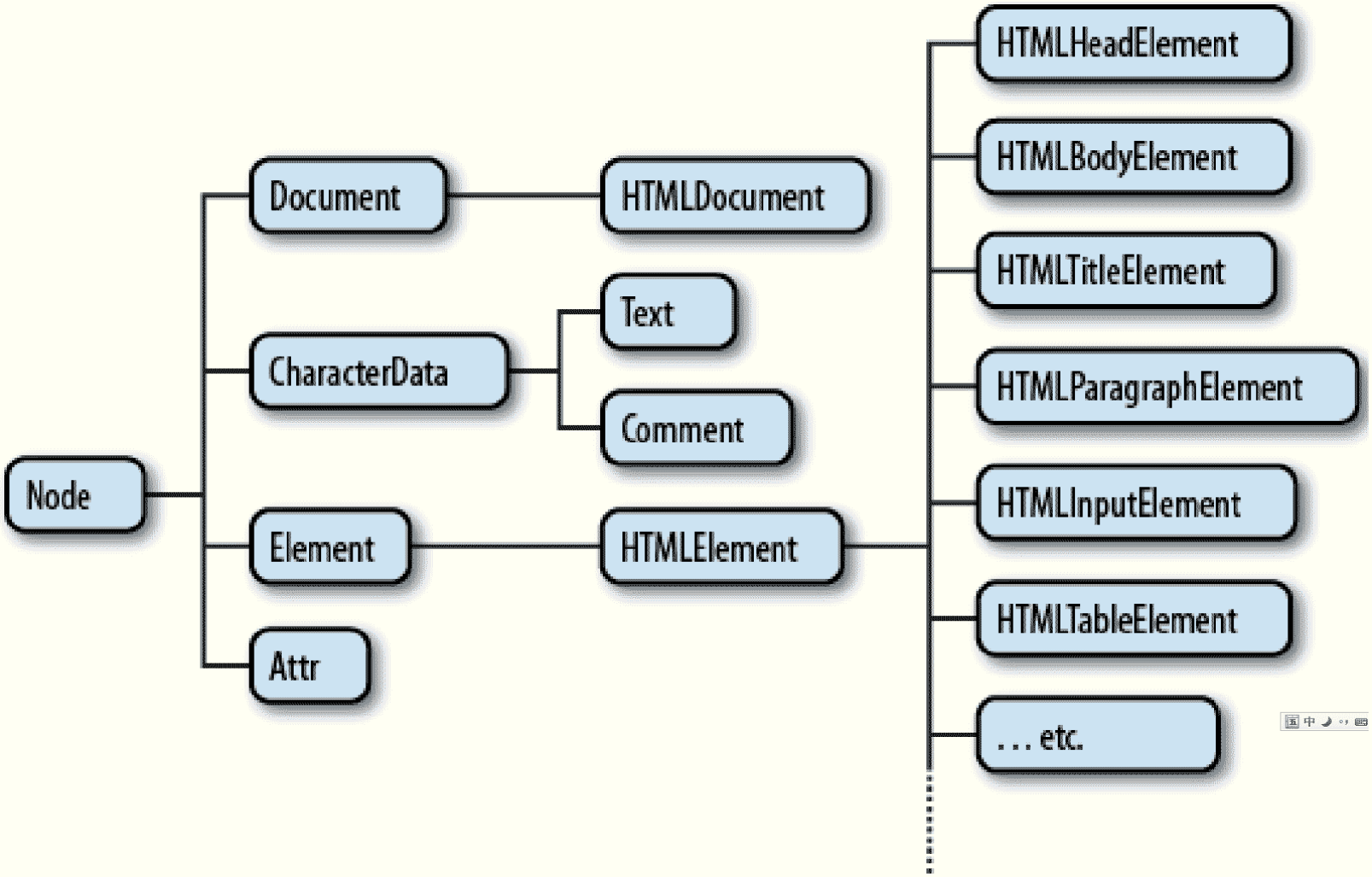
May 21, 2019

DOM概 述

DOM 概述

- DOM (Document Object Model) 文档对象模型
- 是表示和操纵 HTML 文档和 XML 文档的基本编程接口
- 使用 DOM 操纵 HTML 文档和 HTML 元素
  - 获取代表 HTML 标记的 Element 元素
  - 操纵元素的属性与内容
  - 操纵元素的外观（CSS）
  - 操纵文档的结构：插入与删除元素
  - 处理表单

文档节点的部分层次结构



选 择 元 素

通过 ID 选择元素

`Element document.getElementById(elementId)`

- 根据元素的 ID 属性选择元素，
- 同一 HTML 文档中 ID 必须唯一，ID 区分大小写
- 参数 `elementId` 为字符串类型
- 返回值为 Element 类型的元素，通常是 Element 的子类

```
<div id="div1">Hello World</div>
```

```
var divElement =
↪  document.getElementById('div1');
```

通过 name 属性选择元素

```
NodeList getElementsByName(eleName)
```

- 根据元素的 name 属性选择元素
- 参数 eleName 类型为字符串
- 返回具有指定 `eleName` 属性的元素 (类似) 数组
  - 返回值类型为节点列表对象 `NodeList`，类似数组，有 `length` 属性，可用 `[]` 索引，但不可更改

getElementsByName 例子

```
<input type='radio' name='fav_color'
↪  value='red'></input>  红色
<input type='radio' name='fav_color'
↪  value='blue'></input>  蓝色
```

```
// 获取所有 name 属性为 fav_color 的元素
var buttons =
↪  document.getElementsByName('fav_color');
// 遍历获取到的元素
for(i = 0; i < buttons.length; i++) {
  var button = buttons[i];
  console.log(button.value);   //记录按钮的 value
  console.log(button.checked); //记录按钮是否选中
}
```

通过标记名选择元素

`NodeList getElementsByTagName(tagName)`

- 根据标记名选择元素
- 返回对象为 `NodeList` 类型
- 当在 Document 对象上调用时，返回文档中具有指定标记名的节点；
- 当在 Element 对象上调用时，返回 Element 对象子节点中具有指定标记名的节点。

通过标签名选择元素例子

```html
<div id='dv1'>
  <ul>
    <li>梅花</li>  <li>茉莉花</li>
  </ul>
</div>
<ol>
  <li>桃花</li>  <li>杜鹃花</li>
</ol>
```

```javascript
// 在 Document 上调用，返回文档中所有 li 元素
var lis1 = document.getElementsByTagName('li');
console.log(lis1.length); // 4
var dv1 = document.getElementById('dv1');
// 在 Element 上调用，返回 dv1 中所有 li 元素
var lis2 = dv1.getElementsByTagName('li');
console.log(lis2.length); // 2
```

选择具有指定 CSS 类的元素

`NodeList getElementsByClassName(classNames)`

- 选择具有指定 CSS 类的元素
- 参数 `classNames` 为字符串类型，可以是单一类名，也可以是*空格分隔*的类列表
- 返回对象为 `NodeList` 类型
- IE9.0 以下不支持

通过 CSS 类选择元素例子

```
<div id='dv1'>
  <ul>
    <li>梅花</li>
    <li class='active'>茉莉花</li>
    <li>桃花</li>
    <li>杜鹃花</li>
  </ul>
</div>
```

```
var activeLiList =
↪  document.getElementsByClassName('active');
var activeLi = activeLiList[0];
console.log(activeLiList.length)   // 1
console.log(activeLi.textContent)  // 茉莉花
```

通过 CSS 选择器选取元素

`NodeList querySelectorAll(cssSelector)` 选择所有匹配选择符的元素 `NodeList querySelector(cssSelector)` 选择第一个匹配选择符的元素

- 可以在 Document 对象和 Element 对象上调用，前者在文档范围内查询，后者在元素的子元素中查询
- 例：`document.querySelectorAll('#dv1 .red')`

通过 CSS 选择器选取元素例子

```
<div id='dv1'>
  <ul>
    <li class='red'>梅花</li>
    <li class='red'>茉莉花</li>
    <li>桃花</li>
    <li>杜鹃花</li>
  </ul>
</div>
<p class='red'>一个段落</p>
```

通过 CSS 选择器选取元素例子-续

```
var dv1 = document.getElementById('dv1');
var redLi1 = dv1.querySelectorAll('.red');
var redLi2 = document.querySelectorAll('#dv1
↪  .red');
var redElements =
↪  document.querySelectorAll('.red');
console.log(redLi1.length);  // 2
console.log(redLi2.length);  // 2
console.log(redLi1[0].innerHTML ===
↪  redLi2[0].innerHTML);  // true
console.log(redElements.length);  // 3
```

文 档 遍 历

Document 对象、Element 对象及表示文本的 Text 对象都是 Node 对象。Node 对象定义了以下重要属性:

- `parentNode` 节点的父节点，Node 类型
- `childNodes` 节点的子节点，NodeList 类型
- `firstChild` 和 `lastChild` 节点的第一和最后一个子节点
- `nextSibling` 和 `previousSibling` 节点的前一个和下一个兄弟节点

- nodeType 节点的类型（9 代表 Document，1 代表 Element，3 代表 Text，8 代表注释，11 代表 DocumentFragment）
- nodeValue（文本和注释节点）的值

注意：文本和注释也是 Node 类型的节点

```
<p id='p1'>
  <a href='http://www.sdjzu.edu.cn'>建筑大学</a>
</p>
```

```
var p1 = document.getElementById('p1');
console.log(p1.firstChild.nodeType); // 3
```

通常我们不关心文本节点，因此可以使用如下属性 Element 类型的子节点：

- `children` 属性：Element 类型子节点数组
- `firstElementChild` 属性：第一个 Element 类型子节点
- `lastElementChild` 属性：最后一个 Element 类型子节点
- `nextElementSibling` 属性：后一个 Element 类型兄弟节点
- `previousElementSibling` 属性：前一个 Element 类型兄弟节点

操 纵 属 性

操纵标签属性

- HTML 属性作为 Element 对象属性

```
<a id='a1' href='https://www.baidu.com'
↪  title='百度' class='someClass'>百度</a>
<img id="img1" src='images/1.jpg'></img>
```

```
var link = document.getElementById('a1');
link.href = 'http://www.sdjzu.edu.cn';
link.title = '打开建筑大学网站';
link.className = 'anotherClass'; // 注意
↪  class 属性在 JavaScript 中为 className
var image = document.getElementById('img1');
image.src = 'images/2.jpg';
```

- Element 对象的属性取决于它所代表的 HTML 元素的属性

操纵标记属性

- 事件处理程序属性作为 Element 对象属性
- 使用 HTML 事件处理属性:

```
<button onclick='fun1();'>OK</button>
```

- 使用 Element 对象的事件处理属性:

```
<button id='btn1'>OK</button>
```

```
var btn = document.getElementById('btn1');
btn.onclick = function() {
    //
}
```

- *推荐使用第二种方式：将行为与外观分离*

操纵标记属性

Element 类型定义了 getAttribute() 和 setAttribute() 分别用于获取和设置自定义属性

```
<input id='txt1' type='text' myPlaceholder='用户
↪  名'></input>
```

```
var textInput = document.getElementById('txt1');
textInput.getAttribute('myPlaceholder');
textInput.setAttribute('myPlaceholder', 'some
↪  text');
```

操纵标记属性-续

也可以用来获取和设置标准属性。例如，下面的两行代码是等效的：

```
textInput.className = 'TextBox';
textInput.setAttribute('class', 'TextBox');
```

检测与删除属性

- boolean hasAttribute(string attrName)
  - 检测命名属性是否存在
- void removeAttribute(string attrName)
  - 删除命名属性
- 例子:

```
<input id='cb1' type='checkbox'
↪  checked></input>
```

```
var cb1 = document.getElementById('cb1');
console.log(cb1.hasAttribute('checked'));
cb1.removeAttribute('checked');
```

操纵元素内容

innerHTML 属性和 innerText 属性

- Element 接口定义了 innerHTML、innerText 和 textContent 属性
- innerHTML 属性：获取和设置元素内部的 HTML 代码
  - 注意：该属性是 innerHTML，而不是 ~~innerHtml~~
- innerText 属性：获取和设置元素内部的文字内容，不包含标记
- textContent 属性: 作用与 innerText 属性基本相同，但 IE8 及以下不支持
- 上述属性均包含子标签的内容代码

例子

```
<div id="div1">
  <h1>标题 1</h1>
  <p>这是一个段落</p>
</div>
```

```
var div1 = document.getElementById('div1');
console.log(div1.innerHTML);
console.log(div1.innerText);
div1.innerHTML += '<p> 另一个段落 </p>';
```

创建、插入与删除节点

创建节点

- 使用 Document 对象的 createElement() 方法创建新的 Element 节点
  - Element createElement(string tagName)
  - 例子：`var e = document.createElement('div');`
- 使用 Node 对象的 cloneNode() 方法复制节点
  - Node cloneNode(boolean deep)
  - 当 deep 参数为 true 时，递归地复制所有子节点，否则只执行浅复制

插入节点

- 使用 appendChild() 为元素插入子节点
  - Node appendChild(Node newChild)
  - 在需要插入的 Element 节点上调用
  - 新插入的节点成为最后一个子节点

```
<div id='div1'></div>
```

```
var div1 = document.getElementById('div1');
var p = document.createElement('p');
p.innerText = '这是新插入的 p 元素';
div1.appendChild(p);
```

插入节点

- 使用 `insertBefore()` 在元素之前插入节点
  - `Node insertBefore(Node newChild, Node refChild)`
  - 在待插入 Element 节点的父节点上调用
  - 将 `newChild` 插入到 `refChild` 之前

元素之前插入节点 - 例

```
<ul id='ul1'>
  <li></li>
  <li id='li2'></li>
  <li></li>
</ul>
```

```
var ul1 = document.getElementById('ul1');
var li2 = document.getElementById('li2');
var newLi = document.createElement('li');
ul1.insertBefore(newLi, li2);
```

注意

- 注意：在使用 appendChild() 和 insertBefore() 时，如果要插入的节点是文档中已经存在节点，则会先将节点在原位置删除，再在新位置插入。

删除节点

- 使用 removeChild() 删除子节点
  - Node removeChild(Node oldChild)
  - 在父节点上调用
  - 删除 oldChild 节点

```
<ul id='ul1'>
  <li>1</li>
  <li id='li2'>2</li>
  <li>3</li>
</ul>
```

```
var ul1 = document.getElementById('ul1');
var li2 = document.getElementById('li2');
ul1.removeChild(li2);
```

替换节点

- 使用 `replaceChild()` 替换子节点
  - `Node replaceChild(Node newChild, Node oldChild)`
  - 在父节点上调用
  - 使用 `newChild` 替换 `oldChild` 节点

替换节点 - 例

```
<ul id='ul1'>
  <li>1</li>
  <li id='li2'>2</li>
  <li>3</li>
</ul>
```

```
var ul1 = document.getElementById('ul1');
var li2 = document.getElementById('li2');
var newLi = document.createElement('li');
newLi.innerText = '新节点';
ul1.replaceChild(newLi, li2);
```

更改元素的CSS样式

style 属性设置内联样式

- 使用元素的 `style` 属性设置*内联样式*
- 命名规则与 CSS 不同：去掉连字符后面的，连字符后面的第一个字母大写，另外 JavaScript 中的保留字需加 `css` 前缀
- 如：CSS 中 `font-size`，在 JavaScript 对应为 `fontSize`；CSS 中 `float`，在 JavaScript 中为 `cssFloat`
- 所有属性的取值均为字符串，需加引号

```
var p = document.createElement('p');
p.style.color = 'teal';
p.style.fontSize = '12pt';
p.style.position = 'relative';
p.style.left = '300px'; // 注意：值为字符串，加
↪  单位
```

className 属性设置元素的 CSS 类

- 使用元素的 className 属性设置元素的 CSS 类
- 取值可以为单一类名，也可以是空格分隔的列表

```
.Red {
    color: red;
}
.Bold {
    font-weight: bold;
}
```

```
var p = document.createElement('p');
p.className = 'Red Bold';
document.body.appendChild(p);
```

classList 属性控制元素的 CSS 类

使用元素的 classList 属性可以更方便地查询/控制元素的 CSS 类

```
// 添加 CSS 类
e.classList.add('cssClassName');
// 判断是否拥有 CSS 类，返回布尔值
e.classList.contains('cssClassName');
// 删除 CSS 类，如果元素没有这个类，则什么都不做
e.classList.remove('cssClassName');
// 如果元素有这个类，删除它，否则添加它
e.classList.toggle('cssClassName');
```

获取浏览器计算后的元素样式

使用 `window.getComputedStyle()` 方法获取元素经浏览器渲染后的样式

该方法接收两个参数

1. 第一个参数为要获取外观的元素
2. 第二个参数通常为 `null` 或空字符串，但也可以是“:before”、“:after”、“:first-line”或“:first-letter”等

```
var title =
↪  document.getElementById('section1title');
var titlestyles =
↪  window.getComputedStyle(element, null);
```

设置和移除事件处理程序

- 使用 `addEventListener()` 方法为元素注册事件处理程序
- `Element.addEventListener(string eventName, function callback)`
- 使用 `removeEventListener()` 移除事件处理程序
- `Element.removeEventListener(string eventName, function callback)`

示例

```
var btn = document.getElementById('btn1');
// 注册事件处理函数，当单击事件发生在 btn 元素时，
// 执行 handleClick 函数
btn.addEventListener('click', handleClick);
// 移除事件处理函数
btn.removeEventLinstener('click', handleClick);

// 事件处理函数
function handleClick(event) {
    //...
}
```